

Spec Driven Development · Projeto 1

Controle de Gastos Pessoais

Relatório da sessão — esqueleto do projeto e implementação das Aulas 1 e 2
(transações: registrar, listar, editar, excluir e validar)

Python 3.11+

FastAPI

SQLite

Pydantic

CONTEXTO

De onde partimos

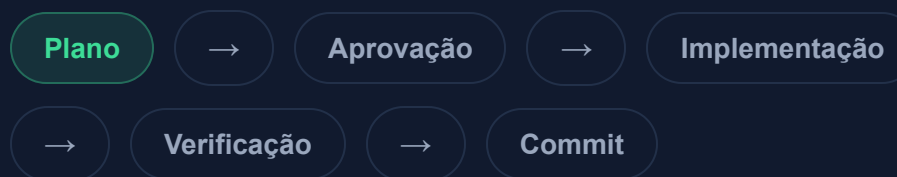
JÁ EXISTIA

- `specs/` com as specs das 6 aulas do projeto
- `docs/prompts-projeto1-controle-gastos.md`, o roteiro de prompts da aula
- `.claude/commands/implementar-spec.md`, o slash command `/implementar-spec`
- Nenhum código da aplicação ainda

OBJETIVO DA SESSÃO

- Criar o esqueleto do projeto + `CLAUDE.md`
- Implementar a Aula 1 (`specs/aula-1-transacoes.md`) e a Aula 2 (`specs/aula-2-crud-validacao.md`)
- Verificar critérios de aceite e commitar a cada aula

Fluxo seguido em cada etapa:



PASSO 1

Esqueleto do projeto + CLAUDE.md

ESTRUTURA CRIADA

```
SDD1/  
├─ CLAUDE.md  
├─ requirements.txt  
├─ .gitignore  
├─ app/  
│   ├── main.py → app + startup + handler 422  
│   ├── database.py → conexão + init_db()  
│   ├── models.py → schemas Pydantic  
│   └─ routers/  
├─ docs/ (já existia)  
├─ specs/ (já existia)  
└─ .claude/ (já existia)
```

CLAUDE.MD DOCUMENTA

- Descrição do projeto e stack
- Como rodar: venv + `pip install` + `uvicorn --reload`
- Estrutura de pastas e papel de cada módulo
- Convenções (datas, valores, tipo, erro 422)
- Seção "Decisões registradas" para acumular ao longo das aulas

Convenções e arquitetura

CONVENÇÕES DO PROJETO

- Datas em ISO `YYYY-MM-DD`
- Valores monetários com 2 casas decimais
- `tipo` sempre `"entrada"` ou `"saida"`
- Erros de validação em HTTP 422, corpo `{ "erro": "..."}`
- Banco SQLite criado na inicialização, se não existir

DECISÕES TÉCNICAS

- Acesso via `sqlite3` puro (sem ORM) — controle direto do SQL para filtros dinâmicos e agregações das próximas aulas
- Erro 422 tratado globalmente com um `exception_handler` de `RequestValidationError` — não repetido rota a rota
- `init_db()` chamado no evento `startup` do FastAPI

Registrar e listar transações

ENDPOINTS IMPLEMENTADOS

- `POST /transacoes` → 201 com a transação criada (com `id`)
- `GET /transacoes` → lista ordenada por `data` decrescente

MODELO DE DADOS

- `id` (auto), `data`, `descricao`, `valor`, `tipo`

ARQUIVOS ALTERADOS

- `app/database.py` — tabela `transacoes`, `criar_transacao`, `listar_transacoes`
- `app/models.py` — `TransacaoCreate`, `Transacao`
- `app/routers/transacoes.py` — rotas POST/GET (novo)
- `app/main.py` — registra o router

PASSO 3

Verificação contra os critérios de aceite

CRITÉRIO	RESULTADO
POST /transacoes retorna 201 com o objeto e id	✓ Passou
GET /transacoes lista mais recentes primeiro	✓ Passou
Transações sobrevivem ao reinício do servidor	✓ Passou
Erros de validação em 422 no formato {"erro": "..."}	✓ Passou

Testado ponta a ponta: servidor subido, transações criadas com datas diferentes, servidor reiniciado e histórico conferido, além de 3 cenários inválidos (valor ≤ 0 , tipo inválido, data malformada).

Versionando o slash command

PROBLEMA ENCONTRADO

O repositório tem um `.gitignore` na raiz que ignora `.claude/` em **todo** o repositório — política já existente, usada para não versionar configs locais.

Isso também excluía `SDD1/.claude/commands/implementar-spec.md`, que não é config local: é parte do fluxo de trabalho do projeto.

SOLUÇÃO

Exceção adicionada ao `.gitignore` raiz, escopada só a este projeto:

→ Reabre `SDD1/.claude/`

→ Reignora tudo dentro, exceto `commands/`

Resto do repositório continua ignorando `.claude/` normalmente.

CRUD completo com validação

ENDPOINTS IMPLEMENTADOS

- `PUT /transacoes/{id}` → 200 com o objeto atualizado, ou 404
- `DELETE /transacoes/{id}` → 204, ou 404
- `PUT` reaproveita o schema do `POST` — mesmas regras de validação valem para os dois

VALIDAÇÃO DE ESCRITA

- `valor > 0`, `tipo` e `data` já vinham da Aula 1
- `descricao` não vazia é nova: `field_validator` que dá `strip()` e rejeita string vazia

ARQUIVOS ALTERADOS

- `app/database.py` — `atualizar_transacao`, `remover_transacao`
- `app/models.py` — validador de `descricao` em `TransacaoCreate`
- `app/routers/transacoes.py` — rotas `PUT` / `DELETE` (novas)

PASSO 5

Verificação — Aula 2

CRITÉRIO	RESULTADO
<code>PUT /transacoes/{id}</code> altera os campos e reflete em <code>GET /transacoes</code>	✓ Passou
<code>DELETE /transacoes/{id}</code> remove; buscar depois retorna 404	✓ Passou
<code>valor ≤ 0</code> , <code>tipo</code> inválido, <code>data</code> malformada ou <code>descricao</code> vazia → 422, em <code>POST</code> e <code>PUT</code>	✓ Passou
Operar sobre <code>id</code> inexistente retorna 404 (<code>PUT</code> e <code>DELETE</code>)	✓ Passou

Testado ponta a ponta: criação, edição, listagem, tentativa de operar em id inexistente, 6 cenários inválidos (em POST e PUT) e exclusão seguida de nova tentativa (404).

Aula 2 — o que ficou definido

VALIDAÇÃO REAPROVEITADA

Como `PUT` usa o mesmo schema `TransacaoCreate` do `POST`, todas as regras (incluindo a nova validação de `descricao`) valem para os dois sem duplicar código.

FORMATO DO 404

`PUT` / `DELETE` em `id` inexistente usam o formato padrão do FastAPI (`{"detail": "..."}`).

O `{"erro": ...}` do CLAUDE.md é uma convenção só para 422 de validação — não foi estendida a outros status sem a spec pedir.

HISTÓRICO

Commits desta sessão

[fea4761](#)

Adiciona Aula 2: CRUD completo com validação

4 arquivos alterados, 65 inserções

[2e8f665](#)

Adiciona relatório em PDF da sessão (esqueleto + Aula 1)

1 arquivo alterado

[648b7c7](#)

Versiona o slash command /implementar-spec do projeto SDD1

2 arquivos alterados, 13 inserções

[d9212f5](#)

Adiciona esqueleto do projeto Controle de Gastos e Aula 1 (transacoes)

16 arquivos alterados, 649 inserções

Projeto pronto para uso

RODANDO LOCALMENTE

- Servidor em `http://127.0.0.1:8000`, com `--reload`
- Documentação interativa em `/docs`
- `gastos.db` criado automaticamente, ignorado no Git
- `.venv/` com as dependências instaladas

ENDPOINTS DISPONÍVEIS

POST	<code>/transacoes</code>
GET	<code>/transacoes</code>
PUT	<code>/transacoes/{id}</code>
DELETE	<code>/transacoes/{id}</code>

Aula 3 — Categorias e filtro por categoria

- 1 **Decidir antes de codar:** categoria é obrigatória ou opcional na transação? Filtrar por categoria inexistente retorna lista vazia ou 404?
- 2 **Rodar** `/implementar-spec specs/aula-3-categorias.md` (criar/listar categorias, associar a transações, filtrar por categoria).
- 3 **Verificar** os critérios de aceite item a item e registrar as decisões no `CLAUDE.md`.
- 4 **Commit por aula** — cada incremento fechado vira um commit, spec incluída.